Simulado de POO — Sistema de E-commerce

Programação Orientada a Objetos · Herança, Classes abstratas, Interfaces, Polimorfismo e Exceções

Nome: _____ Data: ____/____/_____ Nota: _____

Instruções e contexto

Você vai construir, questão por questão, o núcleo de uma loja virtual chamada **TechShop**. Cada questão acrescenta uma parte do sistema e usa o que foi feito nas anteriores. Valor total: 10,0 pontos.

- Linguagem: Java (versão 8 ou superior).
- Todos os atributos devem ser `private`, com getters e setters quando necessário.
- Não altere os nomes de classes, métodos e regras de cálculo pedidos: a correção compara as saídas.
- Questões teóricas devem ser respondidas em 3 a 5 linhas.

O problema. A TechShop vende três tipos de item: produtos físicos (que precisam ser entregues), produtos digitais (download) e assinaturas mensais. Clientes montam um carrinho, escolhem uma forma de pagamento e finalizam o pedido. Clientes VIP recebem desconto. O sistema precisa tratar erros como falta de estoque e pagamento recusado.

Questão 1 — Herança (1,5 ponto)

a) (1,0) Crie a hierarquia de pessoas do sistema:

1. `Pessoa`: atributos `nome`, `cpf` e `email`; construtor com os três; método `toString()` que retorne `"Nome: <nome> | CPF: <cpf>"`.
2. `Cliente extends Pessoa`: atributos `endereco` (String) e `pontosFidelidade` (int, começa em 0); método `adicionarPontos(int pontos)`.
3. `ClienteVip extends Cliente`: atributo `percentualDesconto` (double, ex.: 0.10 para 10%); sobrescreva `toString()` acrescentando `" | VIP (10%)"` ao texto da superclasse.
4. `Funcionario extends Pessoa`: atributos `matricula` e `salario`.

Regras: os construtores das subclasses devem usar `super(...)`; o `toString()` de `ClienteVip` deve reaproveitar o de `Cliente` com `super.toString()`.

b) (0,5) Responda: o trecho abaixo compila? Justifique usando o conceito de relação "é um". E o segundo trecho?

```
Cliente c = new ClienteVip("Ana", "111", "ana@mail.com", "Rua A", 0.10);
ClienteVip v = new Cliente("Ana", "111", "ana@mail.com", "Rua A");
```

Questão 2 — Classes abstratas (2,0 pontos)

a) (1,5) Crie a classe abstrata `Produto` e três subclasses concretas. `Produto` (abstrata) deve ter:

- Atributos `codigo`, `nome`, `precoBase` (double) e `estoque` (int).
- Método abstrato `public abstract double calcularPrecoFinal();`

- Método concreto `public String exibirResumo()` que retorne `"<codigo> - <nome>: R$ <precoFinal>"`, chamando `calcularPrecoFinal()` internamente.
- Método concreto `public void baixarEstoque(int quantidade)`.

Subclasses e suas regras de preço:

Classe	Atributo extra	Regra de calcularPrecoFinal()
`ProdutoFisico`	`pesoKg` (double)	precoBase + 10% de imposto
`ProdutoDigital`	`tamanhoMB` (int)	precoBase − 5% de desconto
`Assinatura`	`meses` (int)	precoBase × meses; se meses ≥ 6, aplica 10% de desconto sobre o total

b) (0,5) Responda:

1. Por que `new Produto(...)` não compila? Que vantagem isso traz para o projeto?
2. `exibirResumo()` está em `Produto`, mas chama um método que `Produto` não implementa. Explique por que isso funciona.

Questão 3 — Interfaces (2,0 pontos)

a) (0,5) Crie a interface `Entregavel`:

```
public interface Entregavel {
    double calcularFrete(String cep);
    int prazoEntregaDias();
}
```

Faça `ProdutoFisico` implementá-la (ele continua estendendo `Produto`). Regras: frete = R$ 15,00 + R$ 2,00 por kg; prazo = 7 dias se o CEP começar com "6", senão 12 dias.

b) (1,0) Crie a interface `Pagamento` com os métodos `double calcularValorFinal(double valor)` e `String getDescricao()`, e um método `default` chamado `exibirComprovante(double valor)` que imprima `"Pago via <descricao>: R$ <valorFinal>"`. Implemente:

Classe	Regra de calcularValorFinal
`PagamentoPix`	5% de desconto
`PagamentoBoleto`	valor sem alteração
`PagamentoCartao`	atributo `parcelas`; acima de 3 parcelas, acréscimo de 8%

c) (0,5) Responda:

1. Por que `Entregavel` foi feita como interface, e não como mais uma classe abstrata na hierarquia de `Produto`?
2. Cite duas diferenças entre interface e classe abstrata em Java.

Questão 4 — Polimorfismo (2,0 pontos)

a) (1,0) Crie a classe `CarrinhoDeCompras`, com uma `List<Produto> itens` e os métodos:

- `adicionar(Produto p)` e `adicionar(Produto p, int quantidade)` (sobrecarga: o segundo adiciona o produto `quantidade` vezes).
- `double calcularTotal()`: soma `calcularPrecoFinal()` de todos os itens, **sem** usar `instanceof` nem verificar o tipo do produto.

- `double calcularFreteTotal(String cep)`: soma o frete apenas dos itens que são `Entregavel`.

b) (0,5) Considere as regras da Questão 2. Qual a saída do código abaixo? Explique qual mecanismo decide qual versão de `calcularPrecoFinal()` é executada.

```
Produto p1 = new ProdutoFisico("F01", "Teclado", 100.0, 5, 0.8);
Produto p2 = new ProdutoDigital("D01", "E-book Java", 50.0, 999, 12);
Produto p3 = new Assinatura("S01", "Streaming", 20.0, 999, 6);

for (Produto p : List.of(p1, p2, p3)) {
    System.out.println(p.getNome() + ": " + p.calcularPrecoFinal());
}
```

c) (0,5) A linha `p1.calcularFrete("60000-000");` compila? Se não, reescreva-a para que funcione e explique a diferença entre o tipo da referência e o tipo do objeto.

Questão 5 — Exceções (2,0 pontos)

a) (1,0) Crie e use as exceções do sistema:

1. `EstoqueInsuficienteException extends Exception` (verificada), com um construtor que receba o nome do produto e a quantidade disponível e gere a mensagem `"Estoque insuficiente para <nome>. Disponível: <qtd>"`.
2. `PagamentoRecusadoException extends RuntimeException` (não verificada).
3. Altere `adicionar(Produto p, int quantidade)` do carrinho para declarar `throws EstoqueInsuficienteException` e lançá-la quando `quantidade` for maior que o estoque.
4. No construtor de `Produto`, lance `IllegalArgumentException` se `precoBase` for negativo.
5. Em `PagamentoCartao`, lance `PagamentoRecusadoException` se o valor final passar de um `limite` (novo atributo).

b) (0,5) O teclado tem estoque 3. Qual a saída do trecho abaixo?

```
try {
    carrinho.adicionar(teclado, 10);
    System.out.println("A");
} catch (EstoqueInsuficienteException e) {
    System.out.println("B: " + e.getMessage());
} finally {
    System.out.println("C");
}
System.out.println("D");
```

c) (0,5) Responda: por que o compilador obriga a tratar `EstoqueInsuficienteException`, mas não `PagamentoRecusadoException`? Em que situação você escolheria cada tipo?

Questão 6 — Integração final (0,5 ponto)

Crie a classe `Pedido`, com um `Cliente`, um `CarrinhoDeCompras` e o método `finalizar(Pagamento pagamento)`, que:

1. Calcula o subtotal (itens + frete para o CEP do cliente).
2. Se o cliente for `ClienteVip`, aplica o `percentualDesconto` dele sobre o subtotal.
3. Calcula o valor final com a forma de pagamento recebida e chama `exibirComprovante`.
4. Adiciona ao cliente 1 ponto de fidelidade a cada R$ 10,00 pagos.

5. Usa `try/catch/finally`: captura `PagamentoRecusadoException` imprimindo `"Pagamento recusado"` e, no `finally`, imprime `"Pedido encerrado"` em qualquer caso.

No `main`, monte um cenário com um `ClienteVip`, um item de cada tipo de produto e duas tentativas de compra: uma com `PagamentoPix` e outra com `PagamentoCartao` de 6 parcelas e limite baixo, para forçar a recusa.

Critérios de correção

Questão	Conteúdo	Pontos	O que se avalia
1	Herança	1,5	Uso de extends e super, sobrescrita de toString(), relação "é um"
2	Classes abstratas	2,0	Método abstrato, método concreto que chama o abstrato, regras de preço corretas
3	Interfaces	2,0	Classe que estende e implementa ao mesmo tempo, método default, diferenças teóricas
4	Polimorfismo	2,0	Total sem instanceof, sobrecarga, ligação dinâmica, cast seguro
5	Exceções	2,0	Verificada × não verificada, throws × throw, ordem de execução do finally
6	Integração	0,5	Todas as partes funcionando juntas no cenário pedido
Total		**10,0**	

Anexo — Diagrama de classes (gabarito do professor)

Setas cheias com triângulo: herança (extends). Setas tracejadas com triângulo: implementação de interface. Losango: agregação. Setas tracejadas abertas: dependência (usa / lança).

